Local RAG Chatbot System

Technical Implementation Documentation

Version 2.0

---

1. Project Overview

Objective

Build a fully local Retrieval-Augmented Generation (RAG) chatbot capable of answering employee questions using internal company PDF documents.

The system:

- ingests PDF documents,
- converts them into semantic vector embeddings,
- stores them in a local vector database,
- retrieves the most relevant text fragments,
- and uses an LLM to generate grounded responses.

The chatbot must:

- minimize hallucinations,
- operate with deterministic outputs,
- support source traceability,
- and maintain low infrastructure complexity.

---

2. System Architecture

```
User Question
      ↓
Frontend React Chat UI
      ↓
FastAPI Backend
      ↓
Query Embedding Generation
      ↓
ChromaDB Vector Search
      ↓
Similarity Threshold Filtering
      ↓
Prompt Construction
      ↓
```

```
Claude API Response Generation
       ↓
Streaming Response to Frontend
       ↓
Answer + Source Citations
```

---

3. Technology Stack & Engineering Justifications

Backend Framework

FastAPI

Why FastAPI?

The chat pipeline is highly I/O-bound:

- embedding generation,
- vector retrieval,
- external LLM streaming.

FastAPI provides:

- asynchronous request handling,
- automatic OpenAPI/Swagger documentation,
- strong typing with Pydantic,
- lower latency under concurrent workloads.

Review Defense

“FastAPI was selected because asynchronous request handling prevents blocking during concurrent LLM and embedding operations. The framework also provides automatic API documentation and schema validation, reducing backend maintenance complexity.”

Vector Database

ChromaDB

Why ChromaDB?

Advantages:

- persistent local storage,
- metadata filtering,
- built-in document association,
- zero cloud dependency,
- simple deployment.

Review Defense

“ChromaDB provides persistent vector storage with integrated metadata management. Unlike raw FAISS implementations, it removes the need for maintaining external mapping layers between vectors and document chunks.”

Embedding Model

Sentence Transformers

Model: `all-MiniLM-L6-v2`

Why This Model?

Advantages:

- lightweight,
- runs fully local,
- low latency,
- excellent semantic retrieval for small corpora,
- deterministic behavior.

Review Defense

“The embedding model was intentionally selected for low inference latency and local execution. For a small enterprise corpus, external embedding APIs would introduce unnecessary cost and additional network failure points.”

LLM Provider

Anthropic

Model: Claude 3.5 Sonnet

Why Claude?

Advantages:

- strong instruction adherence,
- reliable summarization,
- reduced hallucination tendency,
- strong context utilization.

Review Defense

“Claude was selected due to its strong instruction-following capabilities and reliable grounded response generation in retrieval-augmented pipelines.”

---

4. Functional Requirements

The system must:

- ingest multiple PDFs,
- extract text from PDFs,
- split text into semantic chunks,
- generate vector embeddings,
- store embeddings persistently,
- retrieve top-k relevant chunks,
- filter low-confidence retrievals,
- generate grounded answers,
- return source references,
- stream responses to frontend,
- handle invalid inputs gracefully.

---

5. Non-Functional Requirements

Scalability

The architecture must:

- support modular component replacement,
- allow future migration to cloud vector databases,
- support increasing document counts.

Reliability

The system must:

- use deterministic generation (`temperature=0`),
- support persistent vector storage,
- prevent hallucinated responses when retrieval confidence is low.

Security

The system must:

- protect API keys via environment variables,
- restrict production CORS domains,
- avoid exposing raw internal documents.

Maintainability

The architecture must separate:

- ingestion logic,
- retrieval logic,
- API logic,
- frontend UI.

---

6. Project Folder Structure

```
sws-rag/
│
├── backend/
│   │
│   ├── app/
│   │   ├── api/
│   │   │   └── routes.py
│   │   │
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   └── logging_config.py
│   │   │
│   │   ├── services/
│   │   │   ├── rag_service.py
│   │   │   ├── embedding_service.py
│   │   │   └── llm_service.py
│   │   │
│   │   ├── ingestion/
│   │   │   └── ingest.py
│   │   │
│   │   └── main.py
│   │
│   ├── chroma_db/
```

```
│   ├── docs/
│   ├── logs/
│   ├── .env
│   └── requirements.txt
│
└── frontend/
    ├── src/
    │   ├── components/
    │   ├── App.jsx
    │   └── main.jsx
    └── index.html
```

---

7. Backend Dependency Installation

requirements.txt

```
fastapi==0.110.0
uvicorn==0.28.0
anthropic==0.21.0
chromadb==0.4.24
sentence-transformers==2.5.1
pymupdf==1.23.26
langchain-textsplitters==0.0.1
python-dotenv==1.0.1
httpx==0.27.0
```

---

8. Development Roadmap

Phase 1 — Environment Setup

Tasks

- Create project directories
- Setup Python virtual environment
- Install dependencies
- Configure `.env`

.env

```
ANTHROPIC_API_KEY=your_key_here
```

Phase 2 — PDF Ingestion Pipeline

Goal

Convert PDFs into vectorized semantic chunks.

Step 1 — Extract Text

Use:

- PyMuPDF

Responsibilities:

- open PDF,
- extract page text,
- validate non-empty extraction.

Step 2 — Semantic Chunking

Improved Chunking Strategy

```
RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", ".", " ", ""]
)
```

Engineering Justification

“Recursive splitting preserves paragraph and sentence boundaries before falling back to raw character segmentation.”

Step 3 — Generate Embeddings

Use batched embedding generation:

```
embedder.encode(chunks)
```

Engineering Justification

“Batch vectorization improves CPU throughput versus sequential embedding execution.”

Step 4 — Store in ChromaDB

Store:

- chunk text,
- metadata,
- ids,
- embeddings.

Metadata Structure

```
{
    "source": doc_name
}
```

Do NOT duplicate chunk text inside metadata.

Step 5 — Idempotent Ingestion

Prevent duplicate insertion:

```
existing = collection.get(ids=[chunk_id])

if existing["ids"]:
    continue
```

Engineering Justification

“Idempotent ingestion prevents duplicate vector accumulation during repeated indexing operations.”

Phase 3 — Retrieval Pipeline

Step 1 — Query Embedding

Transform user question into vector space.

```
query_vector = embedder.encode([user_question]).tolist()[0]
```

Step 2 — Top-K Semantic Retrieval

```
collection.query(
    query_embeddings=[query_vector],
    n_results=4,
    include=["documents", "metadatas", "distances"]
)
```

Step 3 — Similarity Threshold Filtering

Purpose

Prevent low-confidence chunks from polluting the prompt.

```
THRESHOLD = 1.2
```

Filter irrelevant retrievals before prompt assembly.

Engineering Justification

“Similarity threshold filtering prevents semantically weak retrieval fragments from contaminating generation context.”

Step 4 — Context Compilation

Combine validated chunks:

```
compiled_context = "\n\n---\n\n".join(filtered_chunks)
```

Step 5 — Prompt Engineering

System Prompt Objectives

The prompt must:

- enforce grounding,
- prevent hallucinations,
- reject unsupported answers,
- require source traceability.

Prompt Rules

The model must:

1. answer ONLY using retrieved context,
2. reject unsupported questions,
3. avoid external assumptions,
4. cite source documents.

Phase 4 — Claude Integration

Response Generation Settings

```
temperature = 0.0
```

Engineering Justification

“Zero-temperature inference minimizes output variance and improves deterministic enterprise response behavior.”

Streaming Support

Implement:

- Claude streaming API,
- FastAPI StreamingResponse.

Engineering Justification

“Streaming reduces perceived latency and improves conversational responsiveness.”

Phase 5 — API Layer

FastAPI Endpoints

POST `/api/chat`

Input:

```
{
  "question": "How many annual leave days do employees receive?"
}
```

Output:

```
{
  "answer": "...",
  "sources": [
    "LeavePolicy.pdf"
  ]
}
```

Input Validation

Reject:

- blank queries,
- malformed payloads.

CORS Policy

Development:

```
allow_origins=["*"]
```

Production:

- restrict trusted domains only.

Engineering Justification

“Wildcard origins are enabled exclusively for local development acceleration.”

Phase 6 — Observability & Logging

Logging Requirements

Track:

- user query,
- retrieval latency,
- retrieved documents,
- similarity scores,
- LLM latency,
- token usage,
- errors.

Example

```
logging.info(f"User Query: {user_question}")
logging.info(f"Retrieved Sources: {unique_sources}")
```

Phase 7 — Frontend Development

React Frontend Requirements

Features:

- conversational UI,
- streaming responses,
- markdown rendering,
- source citation display,
- loading indicators,
- retry handling,
- responsive layout.

Suggested Components

```
components/
├── ChatWindow.jsx
├── ChatMessage.jsx
├── SourceList.jsx
├── LoadingSpinner.jsx
```

---

9. Error Handling Strategy

Ingestion Errors

Handle:

- corrupt PDFs,
- scanned/image-only PDFs,
- empty extraction.

Retrieval Errors

Handle:

- missing collections,
- embedding failures,
- empty retrieval results.

LLM Errors

Handle:

- API failures,
- rate limits,
- timeout errors.

---

10. Future Scalability Roadmap

OCR Support

Current limitation:

- image-scanned PDFs unsupported.

Future enhancement:

- Tesseract OCR

- PaddleOCR

Pipeline:

```
Scanned PDF
    ↓
OCR Extraction
    ↓
Semantic Chunking
    ↓
Vectorization
```

Hybrid Retrieval

Future improvement:

- combine semantic search + keyword search.

Example:

- BM25 + vector similarity.

Engineering Justification

“Hybrid lexical-semantic retrieval improves exact terminology matching for enterprise policy documents.”

Advanced Retrieval Enhancements

Future upgrades:

- reranking,
- MMR retrieval,
- contextual compression,
- metadata filtering,
- multi-query retrieval.

---

11. Testing Strategy

Unit Tests

Test:

- PDF extraction,
- chunk generation,
- embedding dimensions,
- vector retrieval.

Integration Tests

Test:

- end-to-end query flow,
- streaming response pipeline,
- API contract validation.

User Acceptance Testing

Verify:

- answer correctness,
- source relevance,
- hallucination resistance,
- latency acceptability.

---

12. Expected Limitations

Current limitations:

- no OCR support,
- limited scalability beyond small corpora,
- no authentication,
- no conversation memory,
- semantic retrieval may miss exact terminology.

---

13. Final Review Defense Summary

Key Engineering Decisions

Component	Justification
FastAPI	Async architecture + low latency
ChromaDB	Simple persistent local vector storage
MiniLM Embeddings	Fast local semantic retrieval
Threshold Filtering	Prevent irrelevant prompt pollution
Temperature=0	Deterministic grounded responses
Streaming	Improved perceived responsiveness
Chunk Overlap	Preserve boundary semantics
Logging	Retrieval observability and debugging
Modular Architecture	Scalability and maintainability

---

14. Recommended Build Order

Build Sequence

1. Environment setup
2. PDF extraction
3. Chunking
4. Embedding generation
5. ChromaDB storage
6. Retrieval testing
7. Prompt engineering
8. Claude integration
9. FastAPI endpoint
10. Streaming support
11. Frontend UI
12. Logging & testing
13. Optimization phase

---

15. Final Goal

The completed system should behave like a lightweight internal enterprise knowledge assistant:

- grounded,
- traceable,
- deterministic,
- modular,
- scalable,
- and production-expandable.